Guía Didáctica del Proyecto TD3 para una Prótesis Mioeléctrica

Conceptos, parámetros, fórmulas, errores encontrados y estado actual del proyecto

Proyecto `EMG_Prosthesis_TD3`

23 de marzo de 2026

Resumen

Este documento es una guía didáctica del proyecto de control continuo de una prótesis mioeléctrica con Twin Delayed Deep Deterministic Policy Gradient (TD3). El objetivo es explicar, de forma amplia y pedagógica, qué está resolviendo el proyecto, cómo está organizado el pipeline, qué significa cada término importante, por qué se eligió TD3, cómo evolucionó la función de recompensa desde una versión heredada hasta la formulación actual basada en tracking MSE, y por qué el hallazgo de un bug en el simulador cambió la interpretación de varias corridas previas. También se explican las fórmulas principales del estado, la reward y el remapeo de acción, además de la lectura correcta de las gráficas de entrenamiento y de los tests visuales. La guía termina con una interpretación clara del baseline válido actual, obtenido con una corrida de 8000 episodios y el checkpoint `Agent7250`, que hoy representa la mejor versión defendible del proyecto en simulación.

Índice

1. Para qué sirve esta guía **3**

2. Qué problema resuelve el proyecto **3**

3. Pipeline general del sistema **3**

4. Glosario de términos clave **4**

4.1. EMG . . . 4
4.2. Features EMG . . . 4
4.3. Estado u observación . . . 4
4.4. Acción . . . 4
4.5. PWM . . . 4
4.6. Reward . . . 4
4.7. Reward shaping . . . 4
4.8. Actor . . . 4
4.9. Crítico . . . 5
4.10. Replay buffer . . . 5
4.11. Tracking MSE . . . 5
4.12. Tracking MAE . . . 5
4.13. ActionL2 . . . 5
4.14. Saturation fraction . . . 5
4.15. Δa L2 . . . 5
4.16. Checkpoint . . . 5

5. Por qué se eligió TD3 **5**

6. Cómo evolucionó la reward **6**
6.1. Reward heredada inicial . . . 6
6.2. Paso a una reward basada en MSE . . . 6
6.3. Reward actual . . . 7

7. Estado actual del problema y por qué tiene 52 dimensiones **7**

8. Por qué se remapea la acción a PWM efectivo **7**

9. El bug del simulador explicado de forma sencilla **7**
9.1. Versión corta . . . 7
9.2. Qué pasaba exactamente . . . 8
9.3. Por qué eso fue tan grave . . . 8
9.4. Cómo se arregló . . . 8

10. Cómo leer las gráficas de entrenamiento **8**
10.1. Qué significa que la reward sea negativa . . . 8
10.2. Qué buscar en la gráfica . . . 8

11. Cómo leer un test visual **9**

12. El baseline válido actual **9**
12.1. Primera versión válida: 2000 episodios . . . 9
12.2. Versión refinada: 8000 episodios . . . 9
12.3. Por qué este resultado es importante . . . 11

13. Qué enseña hoy este proyecto **11**

14. Qué sigue ahora **11**

15. Conclusión **11**

1. Para qué sirve esta guía

Esta guía no pretende reemplazar el reporte técnico del proyecto. Su función es distinta: servir como documento de estudio para entender qué se hizo, por qué se hizo y cómo interpretar el sistema actual. Por eso el texto tiene un tono más pedagógico que de artículo experimental.

En particular, la guía intenta responder preguntas como:

- *Qué problema resuelve exactamente este proyecto?*
- *Por qué se usa TD3 y no otro algoritmo?*
- *Qué significa reward shaping en este contexto?*
- *Por qué la reward inicial era insuficiente?*
- *Qué cambió al pasar a una reward basada en MSE?*
- *Qué fue el bug del simulador y por qué fue tan importante?*
- *Cómo se interpreta el baseline actual de 8000 episodios?*

2. Qué problema resuelve el proyecto

El proyecto busca controlar una prótesis robótica de mano a partir de señales EMG. El objetivo no es solo clasificar gestos discretos, sino lograr una operación continua, es decir, que la prótesis pueda modular su movimiento de forma proporcional y no binaria. Esa idea está alineada con el artículo base del proyecto [Fuertes Bustos et al., 2026] y con la literatura de control mioeléctrico reciente [Chen et al., 2023].

En términos simples:

- la persona activa ciertos grupos musculares;
- esas activaciones se registran como EMG;
- el agente TD3 recibe una representación de esa EMG;
- el agente decide una acción continua para cuatro motores;
- la prótesis se mueve;
- el sistema mide qué tan cerca quedó respecto a una referencia deseada.

3. Pipeline general del sistema

La forma más simple de entender el proyecto es mediante el pipeline de aprendizaje por refuerzo:

$$\text{EMG/observaciones} \rightarrow \text{estado} \rightarrow \text{actor TD3} \rightarrow \text{acción} \rightarrow \text{entorno} \rightarrow \text{reward} \rightarrow \text{buffer} \rightarrow \text{actualización TD3}. \tag{1}$$

En este proyecto, ese pipeline se materializa así:

1. se calcula un vector de 40 features EMG;
2. se añade realimentación del estado mecánico de la prótesis;
3. el actor de TD3 produce cuatro acciones continuas en $[-1, 1]$;

4. el entorno remapea esas acciones a PWM efectivos;
5. el simulador actualiza la posición de la prótesis;
6. se calcula una reward basada en tracking y regularización;
7. la transición se almacena en replay buffer;
8. actor y críticos se actualizan según las reglas de TD3 [Fujimoto et al., 2018, MathWorks, 2026a,b].

4. Glosario de términos clave

4.1. EMG

EMG significa electromiografía. En este proyecto, la EMG representa la actividad muscular del antebrazo y sirve como proxy de la intención motora del usuario.

4.2. Features EMG

No se alimenta la red con la señal cruda. En lugar de eso, se calculan cinco features por cada uno de ocho canales, para un total de 40 dimensiones. Esto reduce ruido y hace el estado más estructurado.

4.3. Estado u observación

Es el vector que ve el agente en cada paso. Hoy el estado válido del proyecto tiene 52 dimensiones y combina EMG actual, encoders, delta de encoders y acción efectiva previa.

4.4. Acción

Es la salida del actor. En este proyecto son cuatro valores continuos, uno por motor, acotados en $[-1, 1]$.

4.5. PWM

PWM es la forma efectiva en que se manda la señal a los motores. Aunque el actor produce valores continuos, el simulador usa niveles efectivos discretos de PWM.

4.6. Reward

Es la señal escalar que dice al agente si una acción fue mejor o peor. No es exactamente "éxito" o "fracaso", sino una medida de cuán deseable fue la transición.

4.7. Reward shaping

Es el diseño de esa reward para que el agente aprenda el comportamiento correcto. Es uno de los puntos más delicados del proyecto [Deng et al., 2025].

4.8. Actor

Es la red que decide la acción.

4.9. Crítico

Es la red que estima qué tan buena es una acción en un estado dado. TD3 usa dos críticos para reducir sobreestimación [Fujimoto et al., 2018].

4.10. Replay buffer

Es una memoria donde se guardan experiencias pasadas para entrenar de forma más estable y menos correlacionada.

4.11. Tracking MSE

Es el error cuadrático medio entre la trayectoria de referencia y la trayectoria de la prótesis. Hoy es la métrica principal del proyecto.

4.12. Tracking MAE

Es el error absoluto medio. Es más fácil de interpretar, pero penaliza menos los errores grandes que MSE.

4.13. ActionL2

Mide qué tan grande es, en promedio, la magnitud de la acción. Es una medida de esfuerzo de control.

4.14. Saturation fraction

Mide qué fracción de acciones efectivas cae cerca del límite del actuador. Si esta métrica es muy alta, el agente está empujando demasiado tiempo al borde.

4.15. Δa L2

Mide qué tanto cambia la acción entre un paso y el siguiente. Es una medida de brusquedad o suavidad.

4.16. Checkpoint

Es un agente guardado durante entrenamiento. El mejor agente no siempre es el último; por eso se auditan checkpoints.

5. Por qué se eligió TD3

TD3 se eligió porque este proyecto es un problema de control continuo. El agente necesita producir acciones reales, no clases discretas. TD3 es adecuado para ese tipo de tareas porque combina tres ideas importantes [Fujimoto et al., 2018, MathWorks, 2026a]:

1. dos críticos para reducir sobreestimación;
2. actualización retrasada del actor para estabilizar aprendizaje;
3. suavizado de la política objetivo para evitar soluciones demasiado frágiles.

De forma intuitiva:

- DDPG puro puede volverse optimista de forma artificial;

- TD3 corrige parte de ese optimismo;
- eso lo vuelve especialmente atractivo para control continuo con ruido y estados complejos.

6. Cómo evolucionó la reward

6.1. Reward heredada inicial

La reward heredada intentaba combinar varias ideas a la vez:

- premiar dirección correcta;
- castigar movimiento inverso;
- incentivar no quedarse quieto;
- penalizar distancia respecto a la referencia;
- penalizar cambios bruscos.

En teoría, eso suena razonable. En la práctica, generaba varios problemas:

- trataba acciones continuas como si fueran discretas;
- mezclaba lógica de dirección con distancia sin una escala clara;
- podía inducir *reward hacking*;
- no quedaba claro qué término dominaba realmente la optimización.

El problema central aquí es metodológico: una reward que parece rica no siempre es una reward buena. Si el agente encuentra una forma de maximizarla sin mejorar el tracking real, la reward no está alineada con el objetivo [Deng et al., 2025].

6.2. Paso a una reward basada en MSE

La primera gran limpieza consistió en pasar a una reward centrada en tracking MSE. La forma más simple de esa versión fue:

$$r_t^{\mathrm{MSE}} = -\frac{1}{4}\sum_{i=1}^{4}\left(e_{t,i}^2 + \lambda_a \tilde{a}_{t,i}^2\right), \tag{2}$$

con:

$$e_{t,i} = \hat{y}_{t,i} - y_{t,i}. \tag{3}$$

Esta reward fue mejor por tres razones:

1. el objetivo principal quedó claro: minimizar error de tracking;
2. el error se volvió interpretable en escala normalizada;
3. el agente dejó de depender tanto de reglas discretas artificiales.

6.3. Reward actual

La versión vigente añade suavidad explícita:

$$r_t = -\frac{1}{4}\sum_{i=1}^{4}\left(e_{t,i}^2 + \lambda_a \tilde{a}_{t,i}^2 + \lambda_{\Delta a}\left(\tilde{a}_{t,i} - \tilde{a}_{t-1,i}\right)^2\right). \tag{4}$$

La interpretación de esta fórmula es simple:

- $e_{t,i}^2$: si el tracking es malo, la reward baja;
- $\lambda_a \tilde{a}_{t,i}^2$: si el agente usa demasiada acción, paga un costo;
- $\lambda_{\Delta a}(\tilde{a}_{t,i} - \tilde{a}_{t-1,i})^2$: si el agente cambia muy bruscamente de acción, también paga un costo.

Esto es importante porque el proyecto ya no busca solo que la prótesis llegue al objetivo. Busca llegar bien, sin gastar acción arbitrariamente ni generar trayectorias violentas.

7. Estado actual del problema y por qué tiene 52 dimensiones

La observación actual es:

$$s_t = \begin{bmatrix} \phi_t \\ \mathrm{enc}_t \\ \Delta\mathrm{enc}_t \\ \tilde{a}_{t-1} \end{bmatrix} \in \mathbb{R}^{52}. \tag{5}$$

Esta fórmula merece explicarse con calma:

- ϕ_t: resume la actividad EMG actual;
- enc_t: le dice al agente en qué posición está la prótesis;
- $\Delta\mathrm{enc}_t$: le dice si se está moviendo y en qué dirección;
- $\tilde{a}_{t-1}$: le recuerda qué acción efectiva aplicó justo antes.

La idea intuitiva es esta: si el agente solo ve EMG actual, le falta contexto motor. Si además ve encoder, cambio de encoder y acción previa, puede razonar mejor sobre la transición actual.

8. Por qué se remapea la acción a PWM efectivo

Aunque el actor produce acciones continuas, el sistema no aplica cualquier valor arbitrario al motor. El entorno remapea a niveles efectivos:

$$\mathcal{U}_{\mathrm{PWM}} = \{0, 64, 96, 128, 160, 192, 224, 255\}. \tag{6}$$

Esto se hizo porque, en la práctica, el simulador y el actuador no distinguen bien valores muy pequeños. Si el agente aprendiera acciones continuas minúsculas, el aprendizaje parecería correcto en matemáticas pero no movería realmente la prótesis.

Este punto es muy importante para entender el proyecto: no basta con que la red "quiera" mover. Hay que alinear esa voluntad numérica con una acción físicamente efectiva.

9. El bug del simulador explicado de forma sencilla

9.1. Versión corta

Durante bastante tiempo, el proyecto parecía entrenar agentes, pero el simulador en realidad no estaba moviendo la prótesis como debía.

9.2. Qué pasaba exactamente

El simulador principal usaba datos de trayectoria desde `fit_C2.mat`. El problema fue que esas trayectorias estaban vacías. Entonces, aunque el agente mandara acción, la prótesis se quedaba casi constante.

Además, dentro de `step`, la acción se reflejaba con un desfase de un paso. Eso significa que el agente actuaba en t, pero el entorno parecía responder en $t + 1$.

9.3. Por qué eso fue tan grave

Porque cambia la interpretación de todo:

- si el entorno no se mueve, no se puede concluir seriamente si la reward es buena o mala;
- tampoco se puede concluir seriamente si 52D, 132D o LSTM mejoran o empeoran;
- las corridas pre-fix sirven como aprendizaje de ingeniería, pero no como benchmark principal.

9.4. Cómo se arregló

Se hicieron dos cambios:

1. cuando `ws` está vacío, el simulador usa `pattern_curve.mat` como respaldo;
2. el tiempo del episodio se adelanta antes de leer la prótesis, para que la acción afecte el paso actual.

La versión simple de la conclusión es: antes el agente aprendía en un mundo parcialmente roto; después del fix, empezó a aprender en un entorno físicamente coherente.

10. Cómo leer las gráficas de entrenamiento

La ventana de entrenamiento de MATLAB muestra tres curvas importantes [MathWorks, 2026c,d]:

- **EpisodeReward**: la recompensa total de cada episodio individual;
- **AverageReward**: promedio móvil del reward, útil para ver tendencia;
- **EpisodeQ0**: estimación del retorno esperable al inicio del episodio según el crítico.

10.1. Qué significa que la reward sea negativa

No significa necesariamente que el entrenamiento esté mal. En este proyecto la reward está formulada como costo negativo:

$$r_t \leq 0. \tag{7}$$

Por tanto, mejorar significa acercarse a cero, no necesariamente volverse positivo.

10.2. Qué buscar en la gráfica

Se busca:

- que `AverageReward` suba de forma sostenida;
- que `EpisodeReward` siga siendo ruidosa, pero con mejor centro;
- que `EpisodeQ0` no se separe de forma absurda del reward observado.

11. Cómo leer un test visual

En los plots por episodio del proyecto:

- la línea naranja es la referencia;
- la línea azul es la respuesta de la prótesis;
- las acciones anotadas muestran qué está mandando el agente.

La pregunta central no es solo *si la azul llega*, sino:

- si acompaña de forma progresiva la referencia;
- si evita quedarse plana;
- si lo logra sin saturarse demasiado tiempo.

12. El baseline válido actual

12.1. Primera versión válida: 2000 episodios

Tras el fix del simulador, la primera corrida válida produjo un test con:

- `trackingMSE = 0.052805`,
- `trackingMAE = 0.179093`,
- `actionL2 = 0.674957`,
- `saturationFraction = 0.474999`,
- Δa L2 $= 0.633264$.

Ese resultado fue importante porque por primera vez demostró tracking real.

12.2. Versión refinada: 8000 episodios

La corrida larga de 8000 episodios refinó el baseline. El mejor checkpoint auditado resultó ser `Agent7250`, no `Agent8000`. Ese detalle es muy importante: en RL, el mejor modelo no siempre es el último.

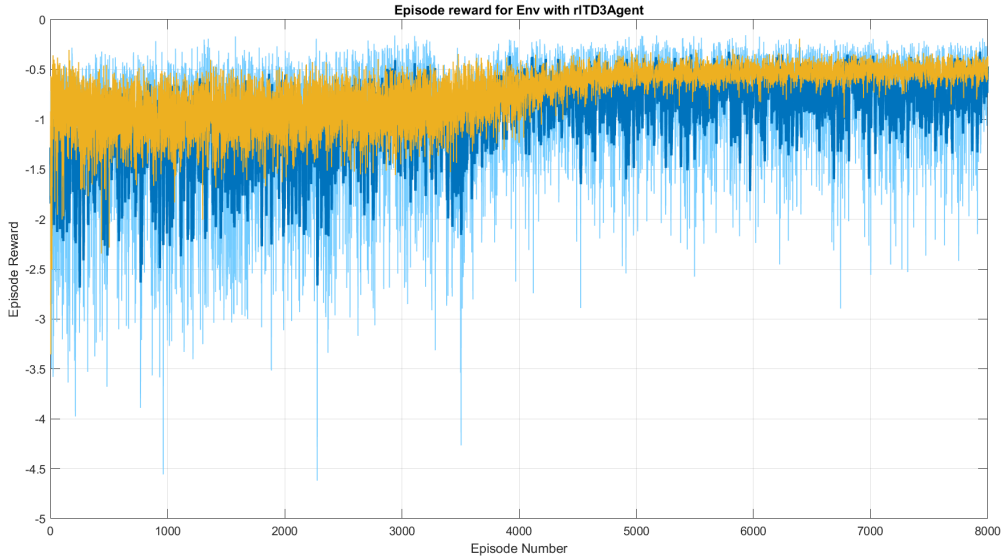

Figura 1: Corrida larga válida de 8000 episodios. La mejora continúa hasta la zona final del entrenamiento.

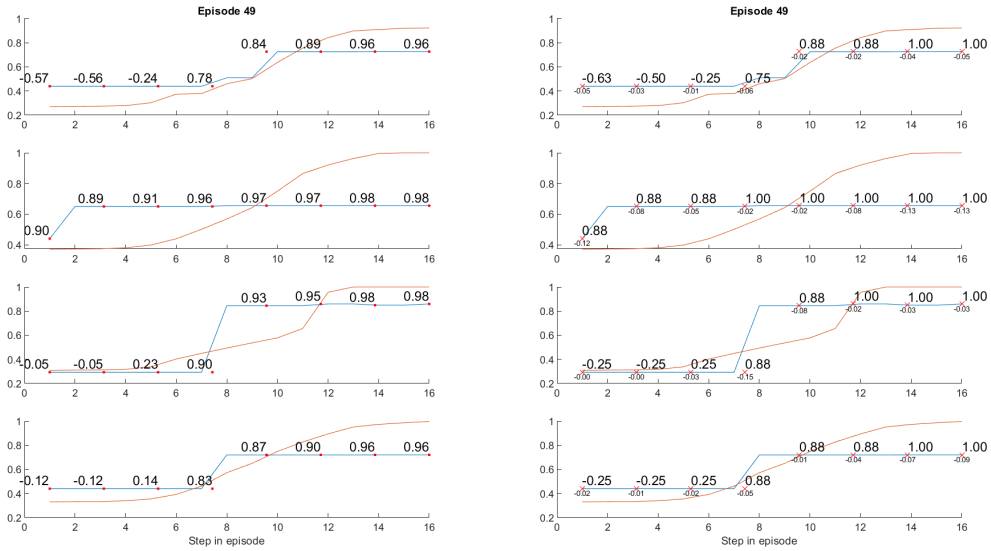

Figura 2: Episodio 49 del test con `Agent7250`. La prótesis sigue la referencia de forma claramente mejor que en el baseline inicial válido.

Métricas del test final de 50 simulaciones con `Agent7250`:

- `trackingMSE = 0.043045`,
- `trackingMAE = 0.160336`,
- `actionL2 = 0.596444`,
- `saturationFraction = 0.392086`,
- Δa L2 $= 0.321385$,
- `|PWM| medio = 178.288566`.

12.3. Por qué este resultado es importante

Porque mejora simultáneamente varias cosas:

- menor error de tracking;
- menor error absoluto;
- menor energía total;
- menor saturación;
- mucha menor brusquedad.

Eso significa que el proyecto ya no solo aprende. Aprende de una forma mejor coordinada.

13. Qué enseña hoy este proyecto

Si alguien quisiera resumir el aprendizaje técnico de todo el proyecto en pocas ideas, serían estas:

1. en problemas de RL para robótica, una reward aparentemente razonable puede estar mal alineada;
2. el *logging* correcto es tan importante como el entrenamiento mismo;
3. una acción continua debe alinearse con la acción física efectiva;
4. si el simulador está mal, las conclusiones sobre la policy también quedan mal;
5. una vez corregido el entorno, TD3 sí puede resolver el tracking de forma convincente en simulación.

14. Qué sigue ahora

El proyecto ya tiene un baseline serio. Por eso la siguiente fase no debería volver a cambiar demasiadas cosas a la vez. Lo más sensato ahora es:

- mantener TD3;
- mantener el estado Markoviano de 52D;
- mantener la reward actual como punto de partida;
- estudiar cómo reducir aún más saturación y esfuerzo sin perder tracking.

En otras palabras, la pregunta ya no es *si el sistema puede aprender*. La pregunta correcta ahora es: *cómo hacer que el control sea igual de preciso, pero más fino y menos agresivo.*

15. Conclusión

Esta guía deja una idea central muy clara: el proyecto no avanzó por una sola gran idea, sino por una secuencia de correcciones bien razonadas. Primero se entendió el problema base. Después se auditó el reward. Luego se pasó a una formulación más limpia con MSE. Más tarde se alineó la acción aprendida con la acción efectiva. Finalmente, se descubrió y corrigió el bug del simulador, que era el cuello de botella más serio.

El resultado de esa historia es el baseline actual con `Agent7250`: un agente que, en simulación, sigue la referencia de manera mucho mejor que al inicio del proyecto y lo hace además con menos brusquedad que el baseline válido inicial.

Ese es el mejor punto posible para seguir aprendiendo y mejorando.

Referencias

Ziming Chen, Huasong Min, Dong Wang, Ziwei Xia, Fuchun Sun, and Bin Fang. A review of myoelectric control for prosthetic hand manipulation. *Biomimetics*, 8(3):328, 2023. doi: 10.3390/biomimetics8030328. URL https://doi.org/10.3390/biomimetics8030328.

Zelin Deng, Yunlong Dong, and Xing Liu. Reward shaping in reinforcement learning for robotic hand manipulation. *Neurocomputing*, 638:130204, 2025. doi: 10.1016/j.neucom.2025.130204. URL https://doi.org/10.1016/j.neucom.2025.130204.

Jose Gabriel Fuertes Bustos, Carlos Ayala, Lorena Isabel Barona, Marco E. Benalcazar, and Angel Leonardo Valdivieso. Continuous operation of a myoelectric hand prosthesis using the td3 reinforcement learning algorithm. Technical manuscript provided by the user, 2026. Baseline conceptual article used as historical reference for the project.

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. https://arxiv.org/abs/1802.09477, 2018. TD3 original paper.

MathWorks. rltd3agent. https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rltd3agent.html, 2026a. Accessed: 2026-03-12.

MathWorks. rltd3agentoptions. https://www.mathworks.com/help/reinforcement-learning/ref/rl.option.rltd3agentoptions.html, 2026b. Accessed: 2026-03-12.

MathWorks. Train reinforcement learning agents. https://www.mathworks.com/help/reinforcement-learning/ug/train-reinforcement-learning-agents.html, 2026c. Accessed: 2026-03-12.

MathWorks. rltrainingoptions. https://www.mathworks.com/help/reinforcement-learning/ref/rl.option.rltrainingoptions.html, 2026d. Accessed: 2026-03-12.